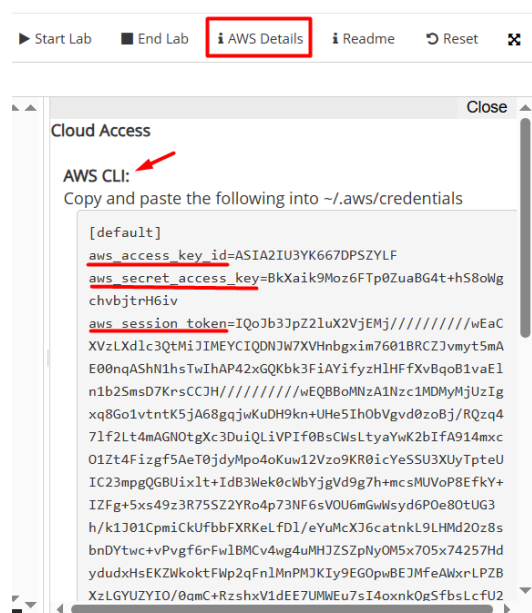


GUÍA DE USO - PROYECTO TPV

En primer lugar al empezar el día de trabajo, inicio la sesión en aws, y entonces necesito ir a “AWS Details” para copiar las credenciales tanto en local, para lanzar comandos como los de terraform a mi laboratorio AWS, como en GitHub para que el actions ejecute correctamente las actualizaciones con mis pushes.



Para ello hice un script “**update-credentials.ps1**” que te pide las tres credenciales y las actualiza en local y el github.

```
PS C:\Users\Damián\Desktop\TPV> .\scripts\update-credentials.ps1
Pega tus credenciales AWS:
aws_access_key_id: ASIA2IU3YK667DPSZYLF
aws_secret_access_key: BkXaIk9Moz6FTp0ZuaBG4t+hS8oWgchvbjtrH6iv
aws_session_token: IQoJb3JpZ2luX2VjEMj////////wEaCXVzLXd1c3QtMiJIMEYCIQDNJW7XVHnbGxim7601BRCZJvmyt5mA
P42xGQKbk3FiAYifzH1HfXvBqoB1vaE1n1b25msD7KrsCCJH////////wEQBB0MNzA1Nzc1MDMyMjUzIg
xq8Go1vtntK5jA68gqjwKuDH9kn+UHe5Ih0bVgvd0zoBj/RQzq471f2Lt4mAGN0tgXc3DuiQLiVPiF0BscWslTyaYwK2bIfA914mxc
01Zt4Fizgf5AeT0jdyMpo4oKuw12Vzo9KR0icYeSSU3XUyTpteUIC23mpgQGBUix1t+IdB3Wek0cWbYjgVd9g7h+mcsMUVoP8Efky+
IZFg+5xs49z3R75SZ2YRo4p73NF6sVOU6mGwWsyd6P0e80tUG3h/k1J01CpmiCkUfbbFXRKeLFD1/eYuMcXJ6catnkL9LHMd2Oz8s
bnDYtwc+vPvgf6rFw1BMCv4wg4uMHJZSZpNyOM5x705x74257HdydudxHsEKZWk0ktFwP2qFn1MnPMJKIy9EG0pwBEJMfAwXrLPZB
JMfAwXrLPZBxZLGYUZYIO/0qmC+RzshxV1dEE7UMWU7sI4oxnk0u5fbsLcfU26DQy9gLmp8bnOyORSmCKST2iJdWVZyjdj8KDjgRi1PYDxQy/jn6p49
3MmXpoPF4Iv4ixe0mH9CI5FrqEETfC0mOnOp6qrXN1jUEIQ6Eepzghp7NteOP8iIpbblVoLZ1Pj2hCrXLCnG4e
== [1/2] Credenciales locales actualizadas!
Set Actions secret AWS_ACCESS_KEY_ID for damimc21/TPV
Set Actions secret AWS_SECRET_ACCESS_KEY for damimc21/TPV
Set Actions secret AWS_SESSION_TOKEN for damimc21/TPV
== [2/2] Secrets de GitHub actualizados!
== Todo listo, ya puedes ejecutar terraform apply y deploy.ps1
PS C:\Users\Damián\Desktop\TPV>
```

El script te pide las tres variables por orden, las guarda y primero recrea las credenciales locales poniendo el [default] con las 3 variables y tra eso las modifica en github gracias a tener vinculado e iniciado sesión con mi github y mediante gh secret actualizamos las credenciales.

```
# =====
# update-credentials.ps1
# =====
Write-Host "Pega tus credenciales AWS:" -ForegroundColor Cyan
$ACCESS_KEY = Read-Host "aws_access_key_id"
$SECRET_KEY = Read-Host "aws_secret_access_key"
$SESSION_TOKEN = Read-Host "aws_session_token"

# Actualizar credenciales locales
$credentials = "[default]`naws_access_key_id = $ACCESS_KEY`naws_secret_access_key = $SECRET_KEY`naws_session_token = $SESSION_TOKEN"
[System.IO.File]::WriteAllText("$env:USERPROFILE\.aws\credentials", $credentials)
Write-Host "[1/2] Credenciales locales actualizadas!" -ForegroundColor Green

# Actualizar secrets en GitHub
$ACCESS_KEY | gh secret set AWS_ACCESS_KEY_ID --repo damimc21/TPV
$SECRET_KEY | gh secret set AWS_SECRET_ACCESS_KEY --repo damimc21/TPV
$SESSION_TOKEN | gh secret set AWS_SESSION_TOKEN --repo damimc21/TPV
Write-Host "[2/2] Secrets de GitHub actualizados!" -ForegroundColor Green

Write-Host "== Todo listo, ya puedes ejecutar terraform apply y deploy.ps1" -ForegroundColor Green
```

Una vez ya tenemos las credenciales listas me voy a la carpeta “infra” y desde allí lanzo el “terraform init” y “terraform apply” para construir toda la estructura aws del proyecto.

Lo que se crea es:

- Una red privada (VPC) donde vive todo lo demás.
- Un repositorio (ECR) donde se guarda la imagen Docker de la aplicación.
- Un bucket (S3) para guardar archivos, como imágenes.
- Una base de datos MySQL (RDS) donde se guardan los datos de la aplicación.
- Un clúster de Kubernetes (EKS) donde finalmente se despliega y ejecuta la aplicación.

El proceso completo tarda varios minutos, sobre todo por la creación del clúster de Kubernetes. Hay que confirmar con “yes” cada vez que la lancemos.

```
module.eks.aws_eks_node_group.main: Still creating... [02m30s elapsed]
module.eks.aws_eks_node_group.main: Still creating... [02m40s elapsed]
module.eks.aws_eks_node_group.main: Still creating... [02m50s elapsed]
module.eks.aws_eks_node_group.main: Still creating... [03m00s elapsed]
module.eks.aws_eks_node_group.main: Still creating... [03m10s elapsed]
module.eks.aws_eks_node_group.main: Still creating... [03m20s elapsed]
module.eks.aws_eks_node_group.main: Still creating... [03m30s elapsed]
module.eks.aws_eks_node_group.main: Creation complete after 3m32s [id=tpv-cluster:tpv-nodes]
Apply complete! Resources: 21 added, 0 changed, 0 destroyed.
```

Una vez creada la infraestructura con Terraform, ejecuto “**deploy.ps1**”. Este script automatiza todo el proceso de subir la aplicación al clúster de Kubernetes, en 7 pasos:

Construye la imagen Docker de la aplicación a partir del código.

Sube la imagen a ECR, el repositorio que se creó con Terraform.

Conecta kubectl al clúster EKS, para poder gestionar Kubernetes desde mi máquina.

Crea los secrets de Kubernetes: coge las credenciales de la base de datos (guardadas en AWS Secrets Manager) y otras claves de la aplicación, y las guarda dentro del clúster para que la app pueda usarlas sin tenerlas escritas en el código.

Ejecuta las migraciones de la base de datos, lanzando un Job de Kubernetes que prepara las tablas necesarias.

Reinicia el despliegue (rolling restart), para que los contenedores empiecen a usar la nueva imagen subida a ECR.

Actualiza el DNS en Cloudflare, apuntando el dominio a la dirección del Load Balancer que ha creado Kubernetes, para que la web sea accesible desde la URL final.

El script usa `$ErrorActionPreference = "Continue"`, así que si un paso falla, no detiene la ejecución completa — sigue intentando los siguientes pasos. Por eso varios comandos llevan comprobaciones manuales “`if ($LASTEXITCODE...)`” en los puntos críticos.

```
# =====
# deploy.ps1 - Construye y despliega la app en AWS/EKS
# Uso: .\scripts\deploy.ps1 (desde cualquier directorio)
# =====

$ErrorActionPreference = "Continue"

# Asegurar que trabajamos desde la raíz del proyecto
Set-Location $PSScriptRoot\..

# — Constantes (deben coincidir con variables.tf y destroy.ps1) —
$PROJECT      = "tpv"
$ACCOUNT_ID   = "705775032253"
$REGION       = "us-east-1"
$CLUSTER_NAME = "$PROJECT-cluster"
$ECR_URL      = "$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com/$PROJECT"
$SECRET_ID    = "$PROJECT/db-credentials"
$K8S_SECRET   = "$PROJECT-secrets"
$K8S_EMAIL_SEC = "$PROJECT-email-secret"
$S3_BUCKET    = "$PROJECT-media-$ACCOUNT_ID"

# — Secretos de aplicación —
$DJANGO_SECRET_KEY = [redacted]
$RESEND_API_KEY     = [redacted]
$CLOUDFLARE_TOKEN   = [redacted]
$CLOUDFLARE_ZONE_ID = [redacted]
$DOMAIN             = "damiantpv.online"

Write-Host ""
Write-Host "  DESPLIEGUE - $($PROJECT.ToUpper())" -ForegroundColor Cyan
Write-Host "  Cluster: $CLUSTER_NAME | Region: $REGION" -ForegroundColor Cyan
Write-Host ""

# — 1. Build de imagen Docker —
Write-Host "== [1/7] Construyendo imagen Docker..." -ForegroundColor Cyan
docker build -t $PROJECT .
```

Build de imagen Docker: “docker build -t \$PROJECT .” construye la imagen usando el Dockerfile en la raíz del proyecto como contexto de build (de ahí el Set-Location \$PSScriptRoot\.. al inicio, para que el script funcione desde cualquier directorio). La imagen queda etiquetada localmente como tpv:latest.

```
# — 2. Push a ECR —————
Write-Host "== [2/7] Autenticando y subiendo imagen a ECR..." -ForegroundColor Cyan
$token = aws ecr get-login-password --region $REGION
docker login --username AWS --password $token "$ACCOUNT_ID.dkr.ecr.$REGION.amazonaws.com"
docker tag "${PROJECT}:latest" "${ECR_URL}:latest"
docker push "${ECR_URL}:latest"

# — 3. Conectar kubectl —————
Write-Host "== [3/7] Conectando kubectl al cluster EKS..." -ForegroundColor Cyan
aws eks update-kubeconfig --region $REGION --name $CLUSTER_NAME

# — 4. Crear/actualizar secrets de Kubernetes —————
Write-Host "== [4/7] Configurando secrets de Kubernetes..." -ForegroundColor Cyan
$dbSecret = aws secretsmanager get-secret-value `
    --secret-id $SECRET_ID --region $REGION --query SecretString --output text | ConvertFrom-Json

kubectl delete secret $K8S_SECRET --ignore-not-found 2>$null
kubectl create secret generic $K8S_SECRET `
    --from-literal=django-secret-key="$DJANGO_SECRET_KEY" `
    --from-literal=database-url="$($dbSecret.url)"

kubectl delete secret $K8S_EMAIL_SEC --ignore-not-found 2>$null
kubectl create secret generic $K8S_EMAIL_SEC `
    --from-literal=resend-api-key="$RESEND_API_KEY"

# — 5. Migraciones —————
Write-Host "== [5/7] Ejecutando migraciones de base de datos..." -ForegroundColor Cyan
kubectl delete job tpv-migrate --ignore-not-found 2>$null
kubectl apply -f k8s/deployment.yaml
kubectl wait --for=condition=complete job/tpv-migrate --timeout=180s
if ($LASTEXITCODE -ne 0) {
    Write-Host "    WARN: El job de migracion no completo en tiempo. Revisa los logs." -ForegroundColor DarkYellow
    kubectl logs job/tpv-migrate
}

# — 6. Rolling restart del deployment —————
Write-Host "== [6/7] Reiniciando deployment y esperando rollout..." -ForegroundColor Cyan
kubectl rollout restart deployment/$PROJECT-app
kubectl rollout status deployment/$PROJECT-app --timeout=300s
```

Push a ECR: “aws ecr get-login-password” pide a AWS un token de autenticación temporal (válido ~12h) para el registry. Ese token se pasa a docker login para autenticar el daemon Docker contra ECR. Después se retaggea la imagen local con la URL completa del repositorio (<account_id>.dkr.ecr.<region>.amazonaws.com/tpv:latest) y se hace docker push.

Conexión de kubectl al clúster: “aws eks update-kubeconfig” añade (o actualiza) una entrada en ~/.kube/config con el endpoint del API server del clúster EKS y configura un exec plugin: cada vez que se llama a kubectl, este invoca la CLI de AWS para generar un token IAM de corta duración, que es como EKS autentica las peticiones (no usa usuario/contraseña).

Secrets de Kubernetes: se lee el secreto de Secrets Manager con aws secretsmanager get-secret-value --query SecretString, que devuelve un JSON (host, usuario, password, url de conexión) y se parsea con ConvertFrom-Json. Como los Secrets de Kubernetes son

inmutables en la práctica (no se pueden "actualizar" un valor con create), primero se borran con `kubectl delete secret --ignore-not-found` y se vuelven a crear de forma imperativa con `kubectl create secret generic --from-literal=...`. Quedan como Secrets de tipo Opaque, montables luego como variables de entorno en los pods.

Migraciones de base de datos: se borra cualquier Job anterior con el mismo nombre (los Jobs de Kubernetes no se pueden relanzar, hay que recrearlos) y se aplica `k8s/deployment.yaml`, que define el Job `tpv-migrate`. `kubectl wait --for=condition=complete --timeout=180s` bloquea el script hasta que el Job termine con éxito o se agote el tiempo; si falla, se muestran los logs del pod para diagnosticar el error.

Rolling restart del deployment: `kubectl rollout restart` añade una anotación con timestamp al pod template del Deployment. Kubernetes detecta el cambio y lanza un rolling update: crea pods nuevos (que ya usan la imagen :latest recién subida a ECR) y va retirando los antiguos progresivamente, sin downtime. `kubectl rollout status --timeout=300s` espera a que termine y devuelve el resultado.

```
# — 7. Actualizar DNS en Cloudflare —————
Write-Host "" [7/7] Actualizando DNS en Cloudflare... -ForegroundColor Cyan
$lbUrl = kubectl get service $PROJECT-service -o jsonpath="{.status.loadBalancer.ingress[0].hostname}"
if (-not $lbUrl) {
    Write-Host "    WARN: No se obtuvo la URL del Load Balancer. DNS no actualizado." -ForegroundColor DarkYellow
} else {
    $headers = @{
        Authorization = "Bearer $CLOUDFLARE_TOKEN"
        "Content-Type" = "application/json"
    }
    $records = Invoke-RestMethod `
        -Uri "https://api.cloudflare.com/client/v4/zones/$CLOUDFLARE_ZONE_ID/dns_records?type=CNAME&name=$DOMAIN" `
        -Headers $headers
    $recordId = $records.result[0].id
    $body = @{ type = "CNAME"; name = $DOMAIN; content = $lbUrl; proxied = $true } | ConvertTo-Json
    Invoke-RestMethod `
        -Uri "https://api.cloudflare.com/client/v4/zones/$CLOUDFLARE_ZONE_ID/dns_records/$recordId" `
        -Method Put -Headers $headers -Body $body | Out-Null
    Write-Host "    DNS actualizado -> $lbUrl" -ForegroundColor Green
}

Write-Host ""
Write-Host ""== Deploy completado!" -ForegroundColor Green
Write-Host ""== URL: https://$DOMAIN" -ForegroundColor Green
```

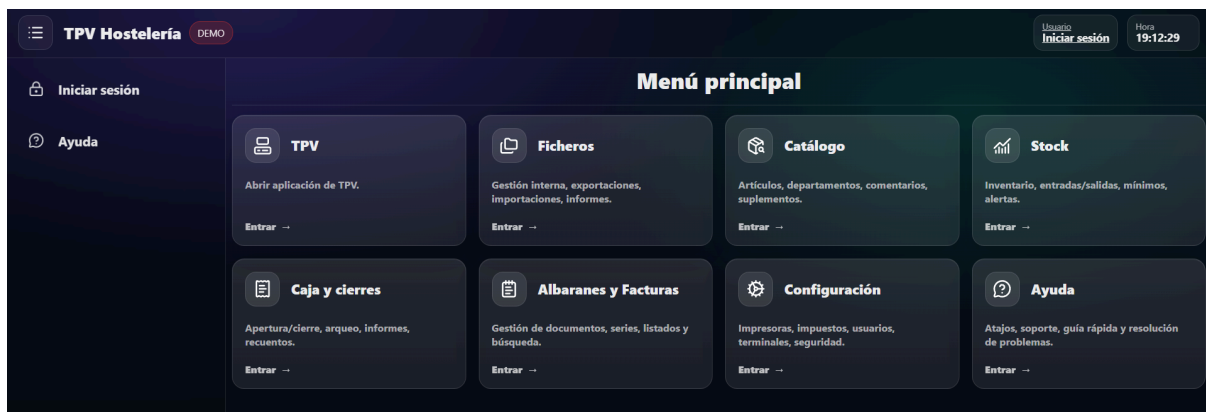
Actualización de DNS en Cloudflare: se obtiene el hostname del Load Balancer que AWS provisionó automáticamente para el Service de tipo LoadBalancer (`kubectl get service ... -o jsonpath`). Con la API REST de Cloudflare se busca primero el ID del registro CNAME existente (`GET /dns_records?type=CNAME&name=...`) y después se actualiza con PUT para que apunte al nuevo hostname del balanceador, con `proxied: true` (el tráfico pasa por el proxy/CDN de Cloudflare en vez de ir directo al Load Balancer).

```

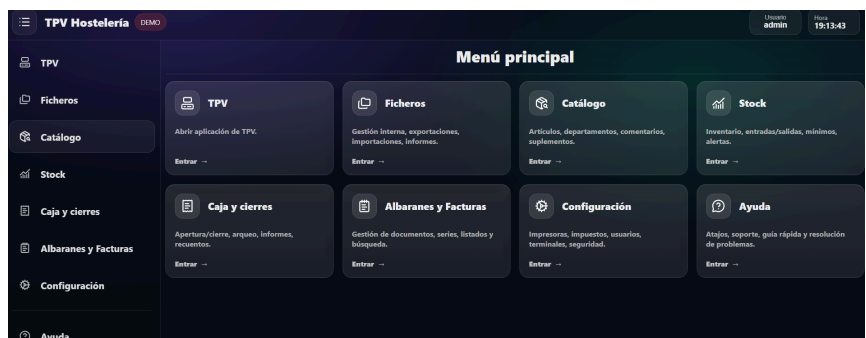
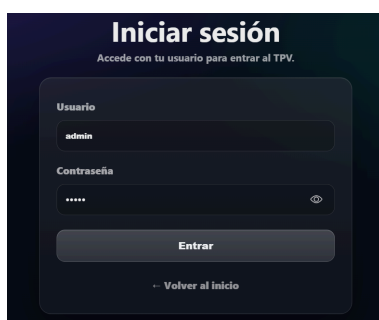
=> CACHED [builder 4/5] COPY requirements.txt requirements-prod.txt ./ 0.0s
=> CACHED [builder 5/5] RUN pip install --no-cache-dir --prefix=/install -r requirements.txt -r requirements-prod.txt 0.0s
=> CACHED [stage-1 4/6] COPY --from=builder /install /usr/local 0.0s
=> [stage-1 5/6] COPY . . 1.4s
=> [stage-1 6/6] RUN DJANGO_SECRET_KEY=build-dummy-key python manage.py collectstatic --noinput 1.7s
=> exporting to image 2.0s
=> exporting layers 0.9s
=> exporting manifest sha256:7ddb23b405cbaadae3bf48dddae33b585c510a11001f0ccc5a2a8949f905ff1 0.0s
=> exporting config sha256:8a4a1409c3f0b5167560a1beb30294c664e4129c1eee3f821d651ab5719dea35 0.0s
=> exporting attestation manifest sha256:d481aa9f15f8a64dfc39260aa2519e193749c986f5a71f2421679a517cb2c939 0.0s
=> exporting manifest list sha256:52e84f0d28dc32850af7b36e41af2f7469acb564074b60f9c7a8e8999edb0435 0.0s
=> naming to docker.io/library/tpv:latest 0.0s
=> unpacking to docker.io/library/tpv:latest 0.0s
== [2/7] Autenticando y subiendo imagen a ECR... 0.9s
WARNING! Using --password via the CLI is insecure. Use --password-stdin.
Login Succeeded
The push refers to repository [705775032253.dkr.ecr.us-east-1.amazonaws.com/tpv]
d8860a0b49c3: Pushed
a7308a408eb1: Pushed
bae41854fae8: Pushed
505d480d2951: Pushed
497a0c371ab8: Pushed
9970f4c20ff1: Pushed
d9821883027d: Pushed
72c03230f136: Pushed
cf5110b4fa79: Pushed
57d30efd7b36: Pushed
latest: digest sha256:52e84f0d28dc32850af7b36e41af2f7469acb564074b60f9c7a8e8999edb0435 size: 856
== [3/7] conectando kubectl al cluster EKS...
Updated context arn:aws:eks:us-east-1:705775032253:cluster/tpv-cluster in C:\Users\Damián\.kube\config
== [4/7] Configurando secrets de Kubernetes...
secret/tpv-secrets created
secret/tpv-email-secret created
== [5/7] Ejecutando migraciones de base de datos...
job.batch/tpv-migrate created
deployment.apps/tpv-app created
service/tpv-service created
cronjob.batch/tpv-exportar-s3 created
job.batch/tpv-migrate condition met
== [6/7] Reiniciando deployment y esperando rollout...
deployment.apps/tpv-app restarted
Waiting for deployment "tpv-app" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "tpv-app" rollout to finish: 1 old replicas are pending termination...
deployment "tpv-app" successfully rolled out
== [7/7] Actualizando DNS en Cloudflare...
DNS actualizado -> a3def8d67e10c4499be4cb1ce9ae6508-1033188852.us-east-1.elb.amazonaws.com
== Deploy completado!
== URL: https://damiantpv.online
PS C:\Users\Damián\Desktop> TPV

```

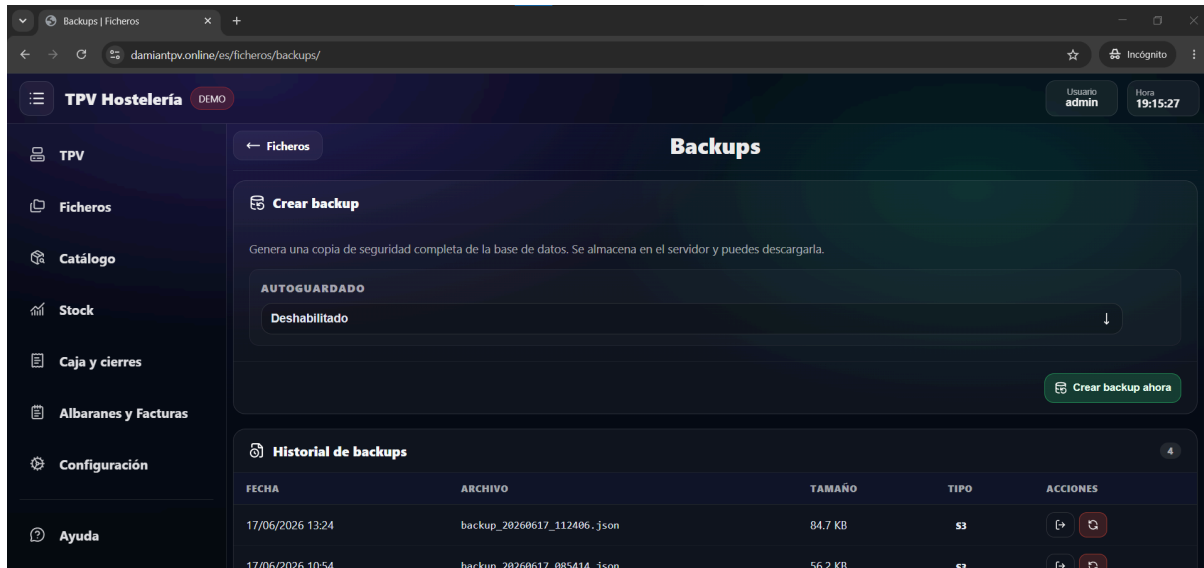
Después de que se ejecute todo esto, ya podremos acceder a la aplicación web alojada en “damiantpv.online”, y gracias a cloudflare tenemos redirección automática de HTTP a HTTPS, protección ante DDoS, etc (Tendremos que iniciar sesión)



Una vez que iniciamos sesión ya tendremos acceso al resto de opciones.



Como podremos comprobar tenemos acceso desde el dominio, y ahora lo que hago para continuar con el proyecto es restaurar una copia de seguridad que previamente guardo en S3 (no se destruye con terraform) para poder guardar todos los datos sin peligro al destruirlo todo y restaurarlos de forma sencilla cuando vaya a trabajar con el proyecto.



Una vez que finalizo la jornada de trabajo lanzo el script destroy.ps1

```
# =====
# destroy.ps1 - Destruye toda la infraestructura AWS (excepto S3)
# Uso: .\scripts\destroy.ps1 (desde cualquier directorio)
# =====

$ErrorActionPreference = "Continue"

# Asegurar que trabajamos desde la raíz del proyecto
Set-Location $PSScriptRoot\..

# — Constantes (deben coincidir con variables.tf) —————
$PROJECT = "tpv"
$REGION = "us-east-1"
$CLUSTER = "$PROJECT-cluster"
$NODEGROUP = "$PROJECT-nodes"
$ECR_NAME = $PROJECT
$VPC_TAG = "$PROJECT-vpc"

function Write-Step($n, $total, $msg) {
    Write-Host ""
    Write-Host "==" [$n/$total] $msg" -ForegroundColor Yellow
}

function Write-Info($msg) { Write-Host " $msg" -ForegroundColor Gray }
function Write-Ok($msg) { Write-Host " OK: $msg" -ForegroundColor Green }
function Write-Warn($msg) { Write-Host " WARN: $msg" -ForegroundColor DarkYellow }
```

Configura que no se detenga si algo falla (`ErrorActionPreference = "Continue"`, lógico al destruir cosas que pueden no existir), se posiciona en la raíz del proyecto, define constantes con los nombres de los recursos (deben coincidir con los que Terraform usó al crearlos) y declara unas funciones de log con color para que la consola se vea ordenada durante la ejecución.

```
# Limpia ENIs disponibles, EIPs y SGs huérfanos de la VPC
function Remove-VpcResiduals($vpcId) {
    if (-not $vpcId -or $vpcId -eq "None") { return }

    $eips = aws ec2 describe-addresses --region $REGION `
        --filters "Name=domain,Values=vpc" `
        --query "Addresses[*].AllocationId" --output text 2>$null
    foreach ($a in (($eips -split "`t") | Where-Object { $_ -and $_ -ne "None" })) {
        $assoc = aws ec2 describe-addresses --region $REGION --allocation-ids $a `
            --query "Addresses[0].AssociationId" --output text 2>$null
        if ($assoc -and $assoc -ne "None") {
            aws ec2 disassociate-address --region $REGION --association-id $assoc 2>$null | Out-Null
        }
        aws ec2 release-address --region $REGION --allocation-id $a 2>$null | Out-Null
        Write-Info "EIP liberada: $a"
    }

    $enis = aws ec2 describe-network-interfaces --region $REGION `
        --filters "Name=vpc-id,Values=$vpcId" "Name=status,Values=available" `
        --query "NetworkInterfaces[*].NetworkInterfaceId" --output text 2>$null
    foreach ($e in (($enis -split "`t") | Where-Object { $_ -and $_ -ne "None" })) {
        aws ec2 delete-network-interface --region $REGION --network-interface-id $e 2>$null | Out-Null
        Write-Info "ENI eliminado: $e"
    }

    $sgs = aws ec2 describe-security-groups --region $REGION `
        --filters "Name=vpc-id,Values=$vpcId" `
        --query "SecurityGroups[?GroupName!='default'].GroupId" --output text 2>$null
    foreach ($sg in (($sgs -split "`t") | Where-Object { $_ -and $_ -ne "None" })) {
        aws ec2 delete-security-group --region $REGION --group-id $sg 2>$null | Out-Null
    }
}
```

Aquí se limpian "huérfanos" que suelen impedir borrar la VPC al final: libera las IPs elásticas (EIPs) asociadas a la VPC, borra las interfaces de red (ENIs) que hayan quedado disponibles/sin usar, y borra los Security Groups personalizados (excluye el "default", que no se puede eliminar). Esto existe porque Terraform a veces no puede destruir la VPC si quedan estos recursos colgando creados fuera de su control directo (por ejemplo, por el propio EKS).

```
# Espera polling hasta que no haya ENIs in-use en la VPC
function Wait-ENIsCleared($vpcId) {
    if (-not $vpcId -or $vpcId -eq "None") { return }
    $maxWait = 300; $elapsed = 0
    do {
        Remove-VpcResiduals -vpcId $vpcId
        $inUse = aws ec2 describe-network-interfaces --region $REGION `
            --filters "Name=vpc-id,Values=$vpcId" "Name=status,Values=in-use" `
            --query "NetworkInterfaces[*].NetworkInterfaceId" --output text 2>$null
        $inUse = ($inUse -split "`t") | Where-Object { $_ -and $_ -ne "None" }
        if ($inUse.Count -eq 0) { Write-Ok "No quedan ENIs en la VPC."; return }
        Write-Info "$($inUse.Count) ENI(s) aun en uso, esperando... ($elapsed s)"
        Start-Sleep -Seconds 20; $elapsed += 20
    } while ($elapsed -lt $maxWait)
    Write-Warn "Timeout esperando ENIs. Terraform intentara igualmente."
}

# =====
Write-Host ""
Write-Host " DESTRUCCION DE INFRAESTRUCTURA - $($PROJECT.ToUpper())" -ForegroundColor Cyan
Write-Host " Region: $REGION | Cluster: $CLUSTER" -ForegroundColor Cyan

# VPC ID - necesario desde el principio (--region obligatorio)
$vpcId = aws ec2 describe-vpcs --region $REGION `
    --filters "Name=tag:Name,Values=$VPC_TAG" `
    --query "Vpcs[0].VpcId" --output text 2>$null
if ($vpcId -and $vpcId -ne "None") {
    Write-Info "VPC encontrada: $vpcId"
} else {
    Write-Warn "VPC '$VPC_TAG' no encontrada. Puede que ya este destruida."
    $vpcId = $null
}
```


Aquí se hace “polling”, es decir cada 20 segundos llama a Remove-VpcResiduals y comprueba si quedan ENIs en uso en la VPC, hasta un máximo de 5 minutos (300s). Si llega a 0, sigue; si se agota el tiempo, avisa y continúa igualmente (deja que sea Terraform quien lo intente). Esto es porque algunas ENIs (como las del load balancer de EKS) tardan en liberarse tras borrar otros recursos, y si la VPC tiene ENIs en uso, Terraform no puede destruirla.

Después arranca el cuerpo del script: imprime una cabecera con el nombre del proyecto y la región, y busca el ID de la VPC por su tag de nombre. Si no la encuentra, asume que ya está destruida y sigue en vez de fallar.

Primero intenta eliminar recursos de Kubernetes: borra el Deployment/Service definido en k8s/deployment.yaml (kubectl delete -f). Si el clúster ya no está disponible o no hay nada que borrar, no falla, solo avisa. Después espera 30s para dar tiempo a que el controlador cloud de AWS procese la baja del Load Balancer asociado al Service.

Después intenta borrar los balanceadores modernos (NLB/ALB, vía elbv2) y los clásicos (ELB, vía elb), recorriendo todos los que existan en la región. Luego hace polling (igual que antes) hasta que no quede ninguno, máximo 5 minutos.

```
# — 3. Vaciar ECR —————
Write-Step 3 8 "Vaciando repositorio ECR '$ECR_NAME'..."
$exists = aws ecr describe-repositories --region $REGION `
  --repository-names $ECR_NAME `
  --query "repositories[0].repositoryName" --output text 2>$null
if ($exists -eq $ECR_NAME) {
  $imgs = aws ecr list-images --region $REGION --repository-name $ECR_NAME `
    --query "imageIds[*]" --output json 2>$null
  if ($imgs -and $imgs -ne "[ ]") {
    aws ecr batch-delete-image --region $REGION --repository-name $ECR_NAME `
      --image-ids $imgs 2>$null | Out-Null
    Write-Ok "Imagenes ECR eliminadas."
  } else { Write-Info "ECR ya estaba vacio." }
} else { Write-Info "ECR no existe o ya eliminado." }

# — 4. Eliminar EKS Node Group via CLI + WAIT —————
Write-Step 4 8 "Eliminando EKS Node Group '$NODEGROUP'..."
$ngExists = aws eks describe-nodegroup --region $REGION `
  --cluster-name $CLUSTER --nodegroup-name $NODEGROUP `
  --query "nodegroup.status" --output text 2>$null
if ($ngExists -and $ngExists -ne "None") {
  aws eks delete-nodegroup --region $REGION `
    --cluster-name $CLUSTER --nodegroup-name $NODEGROUP 2>$null | Out-Null
  Write-Info "Esperando que las instancias EC2 terminen completamente..."
  aws eks wait nodegroup-deleted --region $REGION `
    --cluster-name $CLUSTER --nodegroup-name $NODEGROUP
  if ($LASTEXITCODE -eq 0) {
    Write-Ok "Node group eliminado. Instancias terminadas."
  } else {
    Write-Warn "Timeout en wait nodegroup-deleted."
  }
} else {
  Write-Info "Node group no existe o ya eliminado."
}
```

Ahora intenta vaciar ECR, comprueba si el repositorio de imágenes Docker existe, y si tiene imágenes dentro, las borra todas (batch-delete-image). Esto es necesario porque Terraform no puede borrar un repositorio ECR que aún contenga imágenes; hay que vaciarlo primero a mano.

Y tras eso a por el Node Group de EKS, borra el grupo de nodos (las instancias EC2 que ejecutan los pods) y espera activamente (`aws eks wait nodegroup-deleted`) a que las instancias terminen por completo antes de seguir. Igual que con el LB, las instancias EC2 del node group bloquean la destrucción de la VPC/subredes si siguen vivas, así que hay que esperar a que desaparezcan antes del siguiente paso.

```
# — 5. Eliminar EKS Cluster via CLI + WAIT —————
Write-Step 5 8 "Eliminando EKS Cluster '$CLUSTER'..."
$clusterExists = aws eks describe-cluster --region $REGION `
  --name $CLUSTER --query "cluster.status" --output text 2>$null
if ($clusterExists -and $clusterExists -ne "None") {
  aws eks delete-cluster --region $REGION --name $CLUSTER 2>$null | Out-Null
  Write-Info "Esperando que el cluster y sus ENIs de control plane se liberen..."
  aws eks wait cluster-deleted --region $REGION --name $CLUSTER
  if ($LASTEXITCODE -eq 0) {
    Write-Ok "Cluster EKS eliminado."
  } else {
    Write-Warn "Timeout en wait cluster-deleted."
  }
} else {
  Write-Info "Cluster EKS no existe o ya eliminado."
}

# — 6. Limpiar ENIs, EIPs y SGs residuales —————
Write-Step 6 8 "Limpiando ENIs, EIPs y Security Groups residuales de la VPC..."
Wait-ENIsCleared -vpcId $vpcId

# — 7. Terraform: EKS (sync estado) + RDS + ECR —————
Write-Step 7 8 "Terraform: destruyendo RDS y ECR (sincronizando estado EKS)..."
Set-Location $PSScriptRoot\..\infra
terraform destroy -auto-approve `
  -target "module.eks" `
  -target "module.rds" `
  -target "module.ecr"
$tf1Result = $LASTEXITCODE
Set-Location $PSScriptRoot
if ($tf1Result -ne 0) {
  Write-Warn "Terraform FASE1 con codigo $tf1Result. Continuando con VPC..."
} else {
  Write-Ok "Terraform FASE1 completado."
}

# Segunda limpieza post-Terraform
Remove-VpcResiduals -vpcId $vpcId
```

Se intenta eliminar el clúster EKS, borra el control plane de EKS y espera (`wait cluster-deleted`) a que termine y libere sus ENIs antes de seguir.

Se limpian residuales otra vez, vuelve a llamar a la función de polling de ENIs/EIPs/SGs que ya vimos, ahora que el cluster y el node group ya no existen, para dejar la VPC realmente limpia antes de que entre Terraform.

Ahora Terraform destruye RDS, ECR y EKS, se mueve a /infra y ejecuta terraform destroy pero solo dirigido a esos tres módulos (-target). La idea es que EKS, RDS y ECR ya fueron "vacíados/parados" a mano en los pasos anteriores, así que ahora Terraform puede sincronizar su estado y borrar limpiamente esos recursos sin bloqueos. Si falla, no detiene el script (avisa y sigue, dejando que se intente igualmente la destrucción de la VPC). Justo después se repite otra vez la limpieza de residuales, porque destruir EKS/RDS con Terraform puede dejar nuevas ENIs huérfanas.

```
# — 8. Terraform: VPC —————
Write-Step 8 8 "Terraform: destruyendo VPC..."
Set-Location $PSScriptRoot\..\infra
terraform destroy -auto-approve -target "module.vpc"
$tf2Result = $LASTEXITCODE
Set-Location $PSScriptRoot

Write-Host ""
if ($tf1Result -eq 0 -and $tf2Result -eq 0) {
    Write-Host "==" Infraestructura destruida correctamente." -ForegroundColor Green
    Write-Host "==" S3 conservado. Para redespargar: terraform apply && .\scripts\deploy.ps1" -ForegroundColor Green
} else {
    Write-Host "==" Destroy con errores (FASE1=$tf1Result, FASE2=$tf2Result)." -ForegroundColor Red
    Write-Host "==" Consola AWS VPC: https://console.aws.amazon.com/vpc/home?region=$REGION#vpcs:" -ForegroundColor Red
}
```

Por último Terraform destruye la VPC, ahora que todo lo que dependía de ella (EKS, RDS, load balancers, ENIs) ya está fuera, Terraform puede borrar la VPC y sus recursos de red (subredes, gateways, etc.) sin bloqueos.